

Mule GitHub File Relay – Project Skeleton

A complete, ready-to-import Mule 4 project skeleton that:

- **Flow 1 (Push):** Watches a local folder and **commits new/changed files to a GitHub repo** via the GitHub REST API.
- **Flow 2 (Pull):** On a schedule, **reads files from the GitHub repo and writes them to another machine's folder** (either a mapped network path using the File connector or an SFTP destination using the SFTP connector).

Works even if Gmail/file-sharing is blocked. GitHub acts as the central hub.

1) GitHub Setup – Step by Step

1.1 Create (or choose) a repository

1. Go to **GitHub** → **New repository**.
2. Name it, e.g., `mule-file-relay`.
3. Keep it **Private** (recommended) or **Public**.
4. (Optional) Initialize with a README so the default branch is created.

1.2 Create a Personal Access Token (PAT)

You can use a **Fine-grained** or **Classic** PAT.

- **Fine-grained:**
 - GitHub **Settings** → **Developer settings** → **Personal access tokens** → **Fine-grained tokens** → **Generate new token**.
 - **Repository access:** Only selected repositories → choose your repo.
 - **Permissions** → **Repository permissions** → **Contents:** set to **Read and write**.
 - Save/copy the token (you won't see it again).
- **Classic:**
 - GitHub **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token (classic)**.
 - **Scopes:** select **repo** (full), or at minimum **repo:status**, **repo_deployment**, **public_repo** with **repo** → **Read/Write** for contents.
 - Save/copy the token.

Keep the token secret. We'll load it via Mule **Secure Properties**.

1.3 Notes/limits to remember

- GitHub **Contents API** is best for files ≤ 1 MB. For larger files, use **Git Data API (blobs)** or **Git LFS**. This skeleton targets ≤ 1 MB per file for simplicity.
- Branch protection rules can block direct commits; if enabled, use a PR workflow or commit to a non-protected branch.

2) Project Structure

```
Mule-GitHub-File-Relay/
├─ pom.xml
├─ mule-artifact.json
├─ src/
│   └─ main/
│       └─ mule/
│           ├── global-config.xml
│           ├── push-to-github.xml
│           └─ pull-from-github.xml
│               └─ resources/
│                   ├── app-dev.properties
│                   └─ secure-dev.properties # holds encrypted token
└─ README.md
```

Import this folder into Anypoint Studio as a Mule project.

3) Properties Files

3.1 src/main/resources/app-dev.properties

```
# Environment label
env=dev

# GitHub Repo details
github.owner=YOUR_GITHUB_USERNAME_OR_ORG
github.repo=mule-file-relay
github.branch=main
# All committed files will be placed under this path in the repo (can be empty)
github.basePath=inbox

# Local source folder to watch (Flow 1)
local.watch.dir=C:/SourceFiles
local.processed.dir=C:/SourceFiles/_processed
```

```

# Remote destination (Flow 2) - choose one approach
# Option A: Write to a mapped/network folder (File Connector)
remote.write.dir=Z:/TargetFolder
# Option B: Write via SFTP (SFTP Connector)
sftp.host=your.sftp.host
sftp.port=22
sftp.username=sftp_user
sftp.password=sftp_password
sftp.remote.dir=/var/target

# Scheduler cron for pull flow (every 5 minutes example)
pull.cron=0 0/5 * * * ?

```

3.2 src/main/resources/secure-dev.properties

```

# Store your PAT here (encrypted via Secure Properties)
# Example shown in plaintext for clarity; encrypt it in real use.
github.token=ghp_yourVerySecretTokenHere

```

Encrypt it! Use Mule Secure Properties Tool to encrypt `github.token`, then reference the encrypted value here. Pass the decryption key at runtime via JVM arg `-Dsecure.key=YOUR_KEY`.

4) Global Config (src/main/mule/global-config.xml)

```

<?xml version="1.0" encoding="UTF-8"?>
<mule xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns="http://www.mulesoft.org/schema/mule/core"
      xmlns:ee="http://www.mulesoft.org/schema/mule/ee/core"
      xmlns:http="http://www.mulesoft.org/schema/mule/http"
      xmlns:file="http://www.mulesoft.org/schema/mule/file"
      xmlns:sftp="http://www.mulesoft.org/schema/mule/sftp"
      xmlns:secure-properties="http://www.mulesoft.org/schema/mule/secure-
properties"
      xsi:schemaLocation="http://www.mulesoft.org/schema/mule/core http://
www.mulesoft.org/schema/mule/core/current/mule.xsd
                        http://www.mulesoft.org/schema/mule/ee/core http://
www.mulesoft.org/schema/mule/ee/core/current/mule-ee.xsd
                        http://www.mulesoft.org/schema/mule/http http://
www.mulesoft.org/schema/mule/http/current/mule-http.xsd
                        http://www.mulesoft.org/schema/mule/file http://
www.mulesoft.org/schema/mule/file/current/mule-file.xsd

```

```

        http://www.mulesoft.org/schema/mule/sftp http://
www.mulesoft.org/schema/mule/sftp/current/mule-sftp.xsd
        http://www.mulesoft.org/schema/mule/secure-properties
http://www.mulesoft.org/schema/mule/secure-properties/current/mule-secure-
properties.xsd">

    <!-- Load plain properties -->
    <configuration-properties file="app-${env}.properties" />

    <!-- Secure properties (for PAT) -->
    <secure-properties:config name="Secure_Properties"
        file="secure-${env}.properties"
        key="${secure.key}"
        algorithm="AES"/>

    <!-- File connector configs -->
    <file:config name="File_Config_Source"/>
    <file:config name="File_Config_Remote"/>

    <!-- SFTP (optional) -->
    <sftp:config name="SFTP_Config"
        host="${sftp.host}" port="${sftp.port}"
        username="${sftp.username}" password="${sftp.password}"/>

    <!-- GitHub HTTP Requester with default headers -->
    <http:request-config name="GitHub_Http" basePath="/repos" baseUrl="https://
api.github.com">
        <http:default-headers>
            <http:header headerName="Authorization" value="#['Bearer ' ++
p('github.token')]'"/>
            <http:header headerName="Accept" value="application/
vnd.github+json"/>
            <http:header headerName="User-Agent" value="MuleSoft-App"/>
            <http:header headerName="X-GitHub-API-Version" value="2022-11-28"/>
        </http:default-headers>
        <http:response-validator>
            <http:success-status-code-validator values="200..299"/>
        </http:response-validator>
    </http:request-config>

</mule>

```

5) Flow 1 – Push Local Files to GitHub (src/main/mule/push-to-github.xml)

What it does - Listens to a local folder (recursive), converts file to Base64, **PUTs to GitHub**. - If the file exists, it **updates** (includes `sha`). If not, it **creates**. - Moves processed file to an archive folder.

```
<?xml version="1.0" encoding="UTF-8"?>
<mule xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns="http://www.mulesoft.org/schema/mule/core"
      xmlns:ee="http://www.mulesoft.org/schema/mule/ee/core"
      xmlns:http="http://www.mulesoft.org/schema/mule/http"
      xmlns:file="http://www.mulesoft.org/schema/mule/file"
      xsi:schemaLocation="http://www.mulesoft.org/schema/mule/core http://
www.mulesoft.org/schema/mule/core/current/mule.xsd
                        http://www.mulesoft.org/schema/mule/ee/core http://
www.mulesoft.org/schema/mule/ee/core/current/mule-ee.xsd
                        http://www.mulesoft.org/schema/mule/http http://
www.mulesoft.org/schema/mule/http/current/mule-http.xsd
                        http://www.mulesoft.org/schema/mule/file http://
www.mulesoft.org/schema/mule/file/current/mule-file.xsd">

  <flow name="push-files-to-github">
    <file:listener config-ref="File_Config_Source"
                  directory="${local.watch.dir}"
                  autoDelete="false"
                  recursive="true"
                  moveToDirectory="${local.processed.dir}"
                  watermarkMode="TIMESTAMP"/>

    <!-- Compute repo path: basePath + relative path (normalize slashes) -->
    <set-variable variableName="repoPath" value="#[
      (p('github.basePath') as String) ++ '/' ++
      ((attributes.relativePath default attributes.fileName) as String)
      replace '\\\\' with '/'
    ]"/>

    <!-- Check if file already exists in repo to get its SHA -->
    <try>
      <http:request config-ref="Github_Http" method="GET"
                  path="#[format('/%s/%s/contents/%s',
p('github.owner'), p('github.repo'), vars.repoPath)]"
                  queryParams="#[{'ref': p('github.branch')}]"/>
      <ee:transform doc:name="Extract existing SHA">
        %dw 2.0
        output application/java
        ---
      </ee:transform>
    </try>
  </flow>
</mule>
```

```

        vars existingSha: payload.sha default null
    </ee:transform>
    <on-error-continue enableNotifications="true" logException="true"
when="#[(error.statusCode default 0) == 404]">
        <set-variable variableName="existingSha" value="#[null]" />
    </on-error-continue>
</try>

<!-- Build JSON payload for GitHub PUT (create/update) -->
<ee:transform doc:name="Build GitHub PUT payload">
    %dw 2.0
    output application/json
    ---
    {
        message: "Mule: add/update \"" ++ vars.repoPath ++ "\"",
        content: write(payload, "application/base64"),
        branch: p('github.branch'),
        // Include sha only if updating
        (vars.existingSha default null) != null ? { sha:
vars.existingSha } : {}
    }
</ee:transform>

<!-- PUT to GitHub -->
<http:request config-ref="GitHub_Http" method="PUT"
    path="#[format('/%s/%s/contents/%s', p('github.owner'),
p('github.repo'), vars.repoPath)]" />

    <!-- Log commit info -->
    <logger level="INFO" message='PUSHED #[attributes.fileName] → GitHub
path: #[vars.repoPath]' />
</flow>
</mule>

```

Tip: If you need to preserve the full folder tree under `github.basePath`, keep `recursive=true` and use `attributes.relativePath`. If you only want flat behavior, use `attributes.fileName` instead.

6) Flow 2 – Pull Files from GitHub to Remote (`src/main/mule/pull-from-github.xml`)

What it does - On a schedule (cron), lists files under `github.basePath` on the repo/branch. - For each file, fetches the Base64 **content** using the Contents API, decodes it, and writes it to your destination. - Choose **File** (mapped drive/UNC) or **SFTP** for the destination.

```

<?xml version="1.0" encoding="UTF-8"?>
<mule xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns="http://www.mulesoft.org/schema/mule/core"
      xmlns:ee="http://www.mulesoft.org/schema/mule/ee/core"
      xmlns:http="http://www.mulesoft.org/schema/mule/http"
      xmlns:file="http://www.mulesoft.org/schema/mule/file"
      xmlns:sftp="http://www.mulesoft.org/schema/mule/sftp"
      xsi:schemaLocation="http://www.mulesoft.org/schema/mule/core http://
www.mulesoft.org/schema/mule/core/current/mule.xsd
                        http://www.mulesoft.org/schema/mule/ee/core http://
www.mulesoft.org/schema/mule/ee/core/current/mule-ee.xsd
                        http://www.mulesoft.org/schema/mule/http http://
www.mulesoft.org/schema/mule/http/current/mule-http.xsd
                        http://www.mulesoft.org/schema/mule/file http://
www.mulesoft.org/schema/mule/file/current/mule-file.xsd
                        http://www.mulesoft.org/schema/mule/sftp http://
www.mulesoft.org/schema/mule/sftp/current/mule-sftp.xsd">

  <flow name="pull-files-from-github">
    <scheduler doc:name="Schedule Pull" expression="${pull.cron}" />

    <!-- List directory contents under github.basePath -->
    <http:request config-ref="GitHub_Http" method="GET"
      path="#[format('/%s/%s/contents/%s', p('github.owner'),
p('github.repo'), p('github.basePath'))]"
      queryParams="#[{'ref': p('github.branch')}]" />

    <!-- Expecting an array of directory items; filter files only -->
    <ee:transform doc:name="Filter files">
      %dw 2.0
      output application/java
      ---
      payload filter ((item) -> (item.type default "") == "file")
    </ee:transform>

    <foreach doc:name="For each file" collection="#[payload]">
      <!-- Get base64 content of the file via Contents API -->
      <http:request config-ref="GitHub_Http" method="GET"
        path="#[format('/%s/%s/contents/%s',
p('github.owner'), p('github.repo'), payload.path)]"
        queryParams="#[{'ref': p('github.branch')}]" />

      <!-- Build object with decoded binary and filename -->
      <ee:transform doc:name="Decode content">
        %dw 2.0
        output application/java
        var fname = payload.name

```

```

        var b64    = (payload.content as String)
        ---
        {
            fileName: fname,
            binary: fromBase64(b64)
        }
    </ee:transform>

    <!-- Choose ONE of the two writers below -->

    <!-- Option A: Write to mapped/network folder (File connector) -->
    <file:write config-ref="File_Config_Remote"
        path="#[p('remote.write.dir') ++ '/' ++
payload.fileName]"
        createParentDirectories="true">
        <file:content><![CDATA[#[payload.binary]]]></file:content>
    </file:write>

    <!-- Option B: Write via SFTP (uncomment to use) -->
    <!--
    <sftp:write config-ref="SFTP_Config"
        path="#[p('sftp.remote.dir') ++ '/' ++
payload.fileName]"
        createParentDirectories="true">
        <sftp:content><![CDATA[#[payload.binary]]]></sftp:content>
    </sftp:write>
    -->

    <logger level="INFO" message='PULLED #[payload.fileName] →
destination written' />
    </foreach>
</flow>
</mule>

```

Private repo? Using the **Contents API** with the same `GitHub_Http` config works for private repos. If you instead use the `download_url` (raw file URL), you'll need to pass the PAT on that host too; using the Contents API (as above) is simplest.

7) `pom.xml` (Maven)

Minimal example (adjust Mule plugin versions to your Studio/runtime). If you have an existing parent POM, integrate accordingly.

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"

```

```

        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://
maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>mule-github-file-relay</artifactId>
    <version>1.0.0</version>
    <packaging>mule-application</packaging>

    <properties>
        <mule.maven.plugin.version>3.7.1</mule.maven.plugin.version>
        <mule.runtime.version>4.6.7</mule.runtime.version>
    </properties>

    <build>
        <plugins>
            <plugin>
                <groupId>org.mule.tools.maven</groupId>
                <artifactId>mule-maven-plugin</artifactId>
                <version>${mule.maven.plugin.version}</version>
                <extensions>true</extensions>
                <configuration>
                    <copyToAppsDirectory>true</copyToAppsDirectory>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>

```

8) mule-artifact.json

```

{
  "minMuleVersion": "4.6.0",
  "secureProperties": [
    "github.token"
  ]
}

```

9) How to Run (Dev)

1. **Encrypt the PAT** in `secure-dev.properties` (recommended):
2. Use the Mule Secure Properties Tool to encrypt the value for key `github.token`.

3. Pass JVM arg when running: `-Dsecure.key=YOUR_ENCRYPTION_KEY`.
 4. Ensure the **local folders** exist:
 5. `C:/SourceFiles` and `C:/SourceFiles/_processed`.
 6. Ensure the **remote destination** is reachable:
 7. **File connector**: map a drive (e.g., `Z:`) or use a UNC path (`\\RemotePC\\TargetFolder`). Run Mule as a user with access.
 8. **SFTP**: ensure host, port, username, password are valid.
 9. **Run the app** from Studio.
 10. **Test Push**: Drop a small file (≤ 1 MB) into `C:/SourceFiles`. Watch Studio console: should log `PUSHED ...` and the file should appear in GitHub under `inbox/`.
 11. **Test Pull**: After the scheduler fires, files in repo `inbox/` should be written to your destination; watch logs for `PULLED ...`.
-

10) Handling Updates & Conflicts

- The **push flow** first tries a `GET /contents/{path}`; if **200**, it extracts `sha` and includes it in the **PUT** body $\rightarrow$ **update**. If **404**, it omits `sha` $\rightarrow$ **create**.
 - If branch protection blocks direct pushes, either:
 - Point `github.branch` to a non-protected branch, or
 - Switch to a PR-based workflow (create a branch + pull request via API).
-

11) Optional Enhancements

- **Recursive pulls**: If you use subfolders under `github.basePath`, list items; for items with `type == "dir"`, call the same listing endpoint recursively and flatten. You can implement a small helper subflow for recursion.
 - **Idempotency**: Cache last-seen `sha`s (e.g., Object Store) and skip unchanged files.
 - **Bigger files**: For files > 1 MB, switch to the **Git Data API (blobs)** or enable **Git LFS**.
 - **Webhook-triggered Pull**: Create a GitHub webhook; expose a Mule HTTP Listener to trigger pulls instantly instead of cron.
-

12) Common Errors & Fixes

- **422 (Unprocessable Entity)**: Likely missing `sha` when updating, or invalid base64.

- **409 (Conflict):** Branch protection or out-of-date `sha` (another commit happened). Re-fetch `sha`.
 - **403 (Forbidden):** Token missing/insufficient scopes or repo access denied.
 - **404 (Not Found):** Wrong `owner/repo/path` or wrong `ref` branch.
 - **Private repo raw download fails:** Use the **Contents API** (as in the pull flow) instead of `download_url`.
-

13) Bonus: cURL Samples (quick sanity checks)

Create/Update a file

```
curl -X PUT
-H "Authorization: Bearer $GITHUB_TOKEN"
-H "Accept: application/vnd.github+json"
-H "X-GitHub-API-Version: 2022-11-28"
https://api.github.com/repos/OWNER/REPO/contents/inbox/hello.txt
-d '{
  "message": "Add hello.txt",
  "content": "SGVsbG8gR2l0SHViIQo=",
  "branch": "main"
}'
```

Get file metadata/content

```
curl -H "Authorization: Bearer $GITHUB_TOKEN"
-H "Accept: application/vnd.github+json"
-H "X-GitHub-API-Version: 2022-11-28"
https://api.github.com/repos/OWNER/REPO/contents/inbox/hello.txt?ref=main
```

14) Checklist

- [] Repo created, PAT issued, Contents permission is **Read/Write**.
 - [] `app-dev.properties` filled with repo, paths, and branch.
 - [] `secure-dev.properties` contains **encrypted** `github.token`.
 - [] Local watch folder exists; remote destination reachable.
 - [] Run app, drop file, verify push and pull logs.
-

15) Adapting to Your Infra

- If the **other system** exposes **SFTP**, prefer the SFTP route for reliability and auditing.

- If it's a Windows share, ensure the Mule runtime service account has share/NTFS permissions and use a UNC path in `remote.write.dir`.
 - For **Linux-to-Linux**, you can also mount the remote folder locally and still use the File connector.
-

That's it! Import, set properties, run, and you have a Mule-based relay using GitHub as the secure middleman.